嫉妒图算法

嫉妒图算法

嫉妒图算法（也称为嫉妒环算法），是一种用于离散物品公平分配的算法，可在多位参与者之间分配若干不可分割的物品，例如遗产、糖果或班级座位

理想情况下，我们希望分配结果是 **无嫉妒（EF）** 的，即每位参与者都认为自己分到的物品组合，优于其他所有参与者的物品组合。但由于物品是离散的、不可分割，无嫉妒分配有时无法实现（例如两个参与者分配一件物品的场景）。嫉妒图算法旨在实现“次优”方案——**至多一件物品无嫉妒（EF1）**：该算法能找到一种分配方式，使得任意一位参与者对另一位参与者的嫉妒程度，不超过其从单件物品中获得的最大边际效用。换句话说，对于任意两位参与者 i 和 j，总能找到一件物品，若将该物品从 j 的组合中移除，i 便不再嫉妒 j

前提假设

嫉妒图算法假设，每位参与者对物品组合都有一个**基数效用函数**。该效用函数必须满足**单调性**（一个物品集合的效用，至少不低于其任意子集的效用），但无需满足**可加性**，换言之，不要求物品之间是相互独立的商品

参与者无需实际上报自己的基数效用，只要他们能对不同的物品组合进行排序即可

算法流程

1. 对所有待分配物品进行任意排序
2. 对仍有未分配的物品时，循环执行以下操作：
    - 确保当前存在**无嫉妒参与者**——即没有任何其他参与者嫉妒的参与者
    - 将下一件物品分配给该无嫉妒参与者

在步骤2中，若不存在无嫉妒参与者，意味着嫉妒图中存在有向环。嫉妒图是一个有向图，图中每位参与者会向自己所嫉妒的所有参与者连出有向边。环可以通过物品组合的**循环交换**来消除。当所有环都被消除后，嫉妒图中必然存在一个入度为 0 的节点，该节点对应的就是无嫉妒参与者

算法最终得到的分配结果不一定是完全无嫉妒的，但一定满足“除一件物品外无嫉妒”。不仅最终分配满足这一性质，每一步的中间分配结果也都满足：因为物品始终被分配给无嫉妒参与者，分配完成后，其他所有参与者的嫉妒程度最多只相差一件物品

运行时间分析

假设待分配物品总数为 m。每分配一件物品，嫉妒图中最多新增 $n-1$ 条边（n 为参与者数量），因此整个流程中最多新增 $(n-1)m$ 条边。每次消除环的操作，至少会移除两条边，因此环消除步骤最多需要执行 $(n-1)m/2$ 次。而寻找环的操作可通过深度优先搜索等算法实现，时间复杂度为 $O(n^2)$。综上，该算法的整体时间复杂度为 $O(n^3m)$

One Item Away

2025ICPC深圳邀请赛D题

```
const int N = 110, M = 510;
int n, m;    // 参与者、物品数量
int tot;     // 当前处理到的状态标号
int val[N][M];  // val[i][j]: 第i位参与者对第j种物品集合的估值
int status[N];  // status[i]: 第i位参与者现在持有的物品集合标号
vector<int> items[M];   // items[i]: 第i种物品集合包含的物品
int din[N]; // 嫉妒图入度
vector<int> e[N];   // 嫉妒图连边

void add_item(int item_id, int agent) {
    tot ++;
    items[tot] = items[status[agent]];
    items[tot].push_back(item_id);
    status[agent] = tot;
}

int query(int agent, int id) {
    int sz = items[id].size();
    cout << "? " << agent << ' ' << sz << ' ';
    for(int i = 0; i < sz; i++)
        cout << items[id][i] << " \n"[i + 1 == sz];
    fflush(stdout);
    int res;
    cin >> res;
    return res;
}

void build_graph() {
    for(int i = 1; i <= n; i++) {
        e[i].clear();
        din[i] = 0;
    }
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
            if(val[i][status[j]] > val[i][status[i]]) {
                e[i].push_back(j);
                din[j] ++;
            }
}

// 分配物品
void assign_item(int item_id) {
    for(int i = 1; i <= n; i++)
```

```
        if(din[i] == 0) {
            add_item(item_id, i);
            for(int j = 1; j <= n; j++)
                val[j][tot] = query(j, tot);
            build_graph();
            break;
        }
}

vector<int> find_cycle() {
    int timecnt = 0, scc_cnt = 0;
    vector<int> dfn(n + 1, 0), low(n + 1, 0), instk(n + 1, 0);
    vector<int> id(n + 1), sizes(n + 1, 0);
    vector<int> stk;

    auto tarjan = [&](auto&& self, int u) -> void {
        dfn[u] = low[u] = ++ timecnt;
        stk.push_back(u);
        instk[u] = 1;
        for(auto v: e[u]) {
            if(!dfn[v]) {
                self(self, v);
                low[u] = min(low[u], low[v]);
            } else if(instk[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(dfn[u] == low[u]) {
            int y;
            scc_cnt ++;
            do {
                y = stk.back();
                stk.pop_back();
                instk[y] = 0;
                id[y] = scc_cnt;
                sizes[scc_cnt] ++;
            } while(y != u);
        }
    };
    for(int i = 1; i <= n; i++)
        if(!dfn[i])
            tarjan(tarjan, i);

    vector<int> sccs, cycle;
    vector<int> inscc(n + 1, 0);
    for(int i = 1; i <= scc_cnt; i++)
        if(sizes[i] > 1) {
```

```
            for(int j = 1; j <= n; j++)
                if(id[j] == i) {
                    sccs.push_back(j);
                    inscc[j] = 1;
                }
            break;
        }
    if(sccs.empty()) return {};

    auto dfs = [&](auto&& self, int u) -> bool {
        stk.push_back(u);
        instk[u] = 1;
        for(auto v: e[u])
            if(inscc[v]) {
                if(instk[v]) {
                    do {
                        int t = stk.back();
                        stk.pop_back();
                        cycle.push_back(t);
                        if(t == v) break;
                    } while(true);
                    return true;
                } else {
                    if(self(self, v))
                        return true;
                }
            }
        stk.pop_back();
        instk[u] = 0;
        return false;
    };
    dfs(dfs, sccs[0]);
    return cycle;
}

// 循环消环
void cyclic_exchange() {
    do {
        vector<int> v = find_cycle();
        if(v.empty()) break;
        int sz = v.size();
        vector<int> temp(sz);
        for(int i = 0; i < sz; i++)
            temp[i] = status[v[i]];
        for(int i = 0; i < sz; i++)
            status[v[i]] = temp[(i + sz - 1) % sz];
        build_graph();
```

```
    } while(true);
}

void solve() {
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        cyclic_exchange();
        assign_item(i);
    }

    vector<int> res(m + 1, -1);
    for(int i = 1; i <= n; i++)
        for(auto j: items[status[i]])
            res[j] = i;
    cout << "! ";
    for(int i = 1; i <= m; i++)
        cout << res[i] << " \n"[i == m];
    fflush(stdout);
}
```